

Computación Gráfica

A large, light blue dashed circle is positioned in the upper right corner of the slide, partially overlapping the dark blue background.

Estrategias Gráficas para Implementar un Espejo

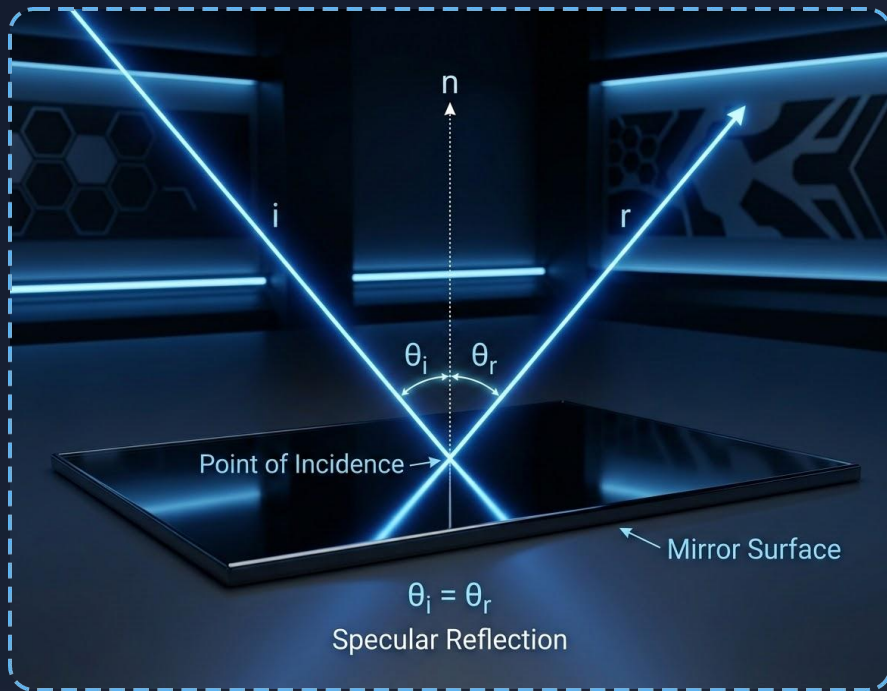
Leonel Pizzuti

Departamento de Ciencias e
Ingeniería de la Computación

¿Qué es un Espejo?

Un espejo es una **superficie lisa** que produce una reflexión especular de la luz.

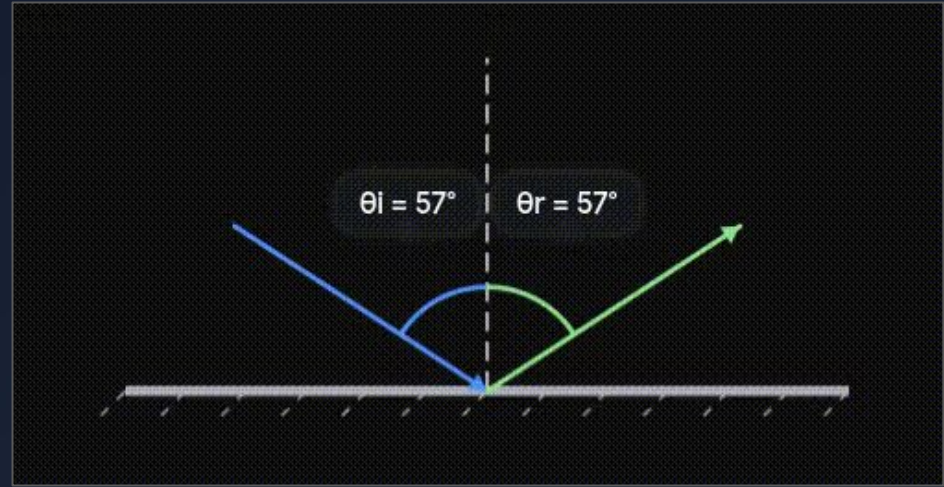
- Cada rayo que impacta genera un vector de salida **calculable y predecible**.
- Sigue las leyes de la óptica geométrica.



Cálculo del Vector

La reflexión se calcula mediante la siguiente relación vectorial:

$$\vec{R} = \vec{I} - 2(\vec{N} * \vec{I})\vec{N}$$



El Desafío: El Acceso a la Información

Calcular el vector **R** no es el problema, sino conocer qué "ve" ese vector.



Procesamiento Aislado

El *Fragment Shader* solo conoce los datos del píxel actual, no la escena completa.



Desconocimiento Global

Al calcular **R**, el shader no sabe qué objeto hay en esa dirección ni su color original.



Costo de Renderizado

Obtener la información visual en **R** exige duplicar geometría o procesar buffers adicionales.

Estrategia 1: Reflejo Planar

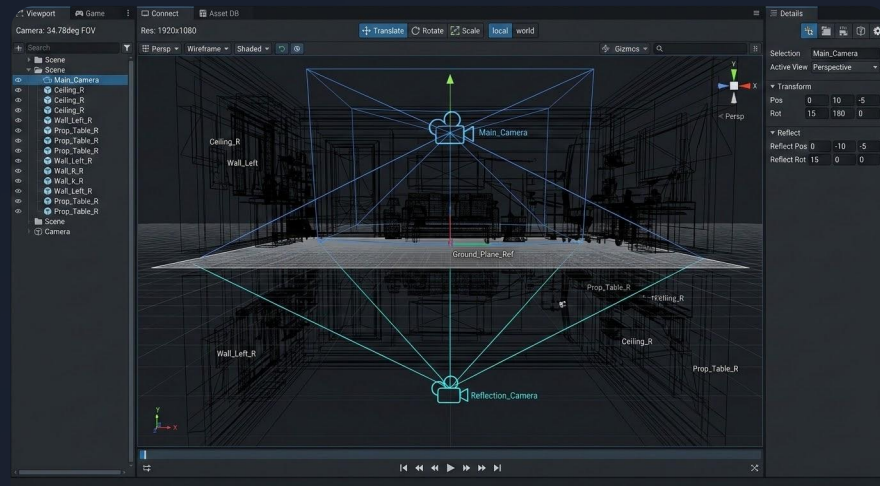
Concepto: Duplicar la cámara principal, invertirla respecto al plano y renderizar en una textura (FBO).

Precisión: Reflejos perfectos, en tiempo real y dinámicos.

Limitación: Funciona exclusivamente en superficies planas.

Costo: Muy alto. Duplica procesamiento de vértices y Draw Calls.

Esquema de Cámaras



Estrategia 2: Cubemaps

Concepto: Capturar el entorno desde un punto central en seis texturas ortogonales (un cubo) y usar el vector R como coordenada 3D.

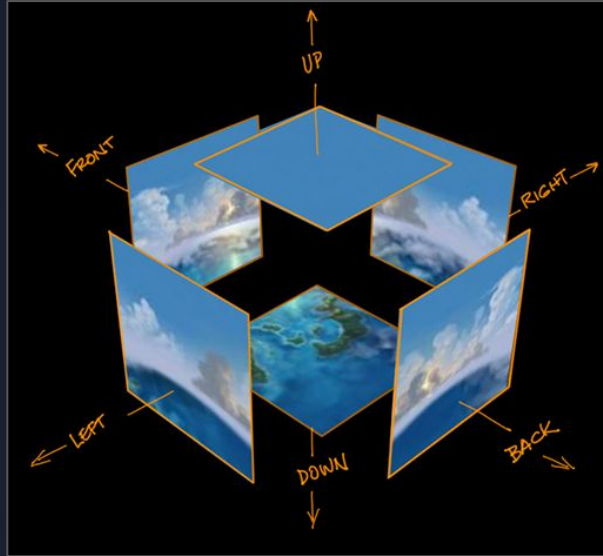
Versatilidad: Reflejos en mallas tridimensionales con topologías curvas y complejas.

Costo GPU: Muy bajo. Es una simple lectura de memoria (texture lookup) por píxel.

Limitación: Generalmente precalculados (Baked). El costo dinámico es inviable.



Estrategia 2: Cubemaps



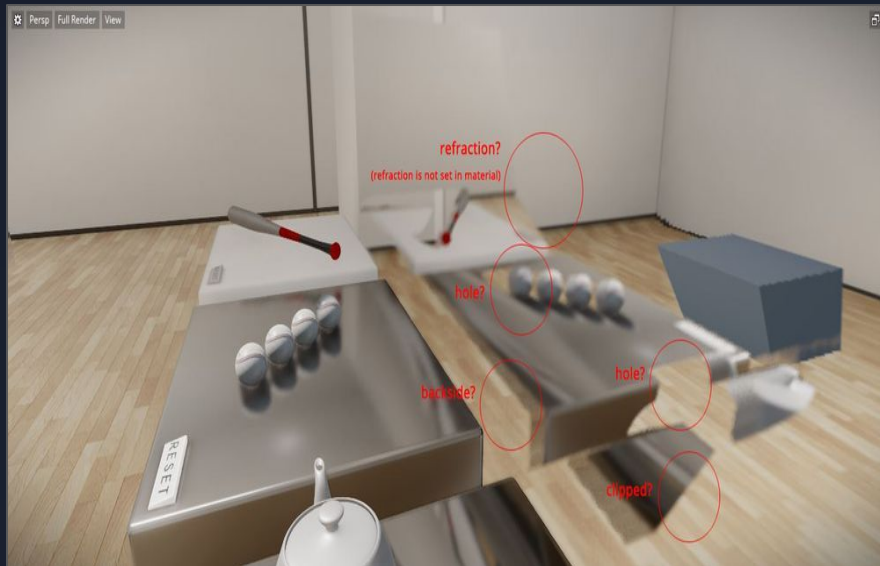
Estrategia 3: Screen Space Reflections

Concepto: Técnica de post-procesamiento que recicla los datos del frame actual (Depth y Color Buffer). Utiliza Ray Marching para encontrar colisiones.

Rendimiento: Excelente relación costo/beneficio. Permite reflejos dinámicos reciclando información ya renderizada.

Independencia: El costo se basa en la resolución de la pantalla, no en la complejidad de los modelos 3D.

Limitación: Solo refleja información visible. Sufre artefactos por oclusión o límites de pantalla (Edges).



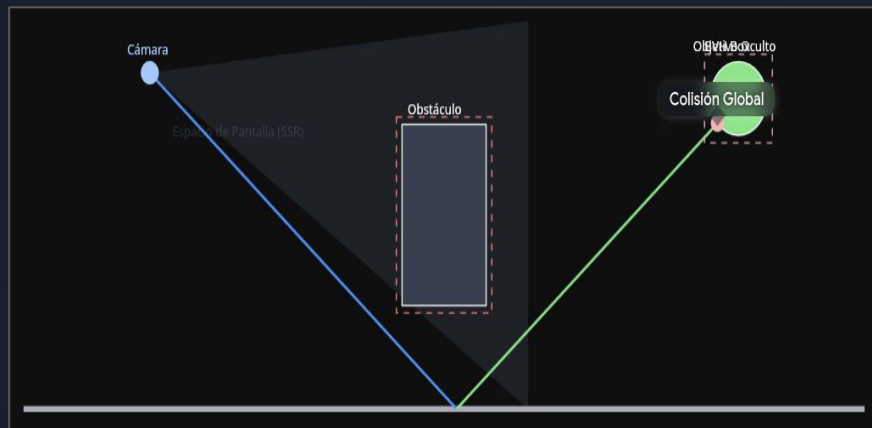
Estrategia 4: Ray Tracing

Concepto: Simulación física del comportamiento de la luz. Se lanzan rayos secundarios hacia la escena global mediante estructuras BVH.

Precisión Absoluta: Físicamente correcto. Soporta cualquier topología y rugosidad perfecta.

Visión Global: Elimina artefactos de oclusión. Refleja objetos fuera de cámara u ocultos.

Costo Arquitectónico: Extremadamente alto. Requiere RT Cores para procesar el árbol BVH.



Comparativa

Estrategia	Precisión Visual	Costo Computacional	Limitación Principal
Reflejo Planar	Perfecta (Plana)	Muy Alto (Doble Draw Call)	Matemáticamente limitado a planos
Cubemaps	Media/Buena	Muy Bajo/Muy Alto	Entornos mayormente estáticos
SSR	Alta	Medio/Bajo (Ray Marching)	Ceguera ante objetos ocluidos
Ray Tracing	Físicamente Correcta	Muy Alto (Exige RT Cores)	Dependencia estricta de hardware

Recursos

Wikipedia: Cube Mapping

https://en-wikipedia-org.translate.google/wiki/Cube_mapping

Unity Manual: Screen Space Reflection

<https://docs.unity3d.com/es/2019.4/Manual/PostProcessing-ScreenSpaceReflection.html>

NVIDIA Developer: Ray Tracing

<https://developer.nvidia.com/discover/ray-tracing>

Google Gemini

Asistente de Inteligencia Artificial

Fin.